UjuziChain - Technical Report

Maxwel Okoth | ALU Low Level Programming Capstone

GitHub Repository
Walkthrough Video Demo
UjuziChain Live Link

Overview

UjuziChain is a decentralized academic credential and engineering competency infrastructure built for the East African tech ecosystem. It anchors university diplomas to Ethereum Sepolia smart contracts, uses client-side SHA-256 hashing for zero-PII storage, and applies a dual-gate screening engine (>= 70% Auto Apply, < 70% Practical Simulation) to connect African developers with regional and global tech recruiters. This report maps the delivered repository to the approved project scope and documents the technical implementation, testing validation, and OOP code quality.

Section 1: Scope Alignment & Feature Validation

Planned Feature	Intended Purpose	Current Status	Evidence
On-Chain Credential Registry	Prevent diploma forgery via public, immutable smart contract records.	Completed	contracts/UjuziChain.sol (Sepolia deployed).
Non-Custodial Address Auth	Self-Sovereign Identity (SSI) with zero password/PII database honeypots.	Completed	frontend/js/auth.js via MetaMask EIP-1193.
Dual-Gate Competency Engine	Balance academic pedigree with practical skills (>=70% Auto Apply, <70% Sim).	Completed	frontend/dashboard.html, UjuziEngine.js.
Zero-Knowledge Doc Hashing	Comply with Kenya Data Protection Act & GDPR by avoiding central PDF storage.	Completed	Client-side SHA-256 generation using window.crypto.subtle.
In-Browser Document Inspector	Allow recruiters to inspect verified certificates without forcing local downloads.	Completed	Irembo-style overlay in frontend/dashboard.html.
East Africa Live Job Ingestion	Ingest real regional developer roles (Nairobi, Kigali,	Completed	Asynchronous API integration via Remotive developer API.

	Kampala, Dar es Salaam).		
W3C Institutional Attestation	Close the trust loop by dispatching auto-verification magic links to universities.	Completed	netlify/functions/send-verification.js & verify-request.html.

Section 2: Technical Product Demonstration

A. Core Functionalities

1. Non-Custodial Wallet Authentication: Candidates log in via MetaMask (EIP-1193), proving ownership of their Ethereum address without sending passwords to a central server.
2. On-Chain Credential Verification: Recruiters input a credential ID which directly queries the Sepolia smart contract (read-only, zero gas) to prove academic legitimacy.
3. Dual-Gate Applicant Screening: Matches >= 70% automatically qualify to apply. Matches < 70% trigger a mandatory interactive technical simulation.
4. W3C Institutional Attestation: New registrations auto-dispatch an email (via Netlify Serverless Functions) to universities to attest hashes on-chain.

B. Technical Requirements

The application relies on Solidity (v0.8.19) and Hardhat for smart contracts deployed on Sepolia (0xA40d...132F). The frontend is vanilla HTML5/JS integrated with ethers.js v6.7.0. Data handling utilizes client-side LocalStorage for non-sensitive session states, and the Web Crypto API for secure client-side SHA-256 document hashing.

C. Testing with Different Inputs

The system exhibits robust error handling. For instance, entering an invalid Sepolia Credential ID in the verification portal safely catches the revert and displays a distinct 'Credential Not Found' state. The dynamic skill array matcher accurately scales bounds between 30% and 95% depending on tag overlaps, dynamically locking or unlocking the 'Apply' button based on exactly a 70% boundary limit.

D. Testing Strategies

1. Algorithmic Unit Testing: We utilize a Node.js assert-based script (test/ujuzi_test.js) targeting the CompetencyAlgorithm. It explicitly tests 100% skill overlap scenarios against empty array edge cases. Results confirmed the floor threshold successfully protects against 0-skill auto-applications.
2. Functional Gate Testing: This strategy validates dual-gate access control across candidate states (Auto Apply, Simulation Required, Simulation Passed, Ineligible). Testing confirmed transitions lock unauthorized users out accurately.

E. Performance & Compatibility

The frontend is fully responsive across mobile, tablet, and desktop viewports, avoiding overflow via strict view locks. It is hosted globally on Netlify's Edge CDN, achieving sub-800ms load times. Network usage is minimized by completely offloading identity custody and database queries to the client browser and free JSON-RPC Ethereum providers.

Section 3: Code Quality & Technical Implementation

Object-Oriented Programming (OOP)

Frontend logic leverages ES6 classes (frontend/js/services/UjuziEngine.js) emphasizing Abstraction and Encapsulation. For example, the CryptographyService encapsulates the native window.crypto.subtle methods, abstracting complex buffer array conversions away from the UI. The SepoliaRegistryClient acts as a dedicated class to instantiate ethers.js providers, achieving strict separation of concerns.

Algorithms & Problem Solving

1. Cryptographic Document Hashing: Instead of uploading PII-sensitive PDFs to an S3 bucket, a local ArrayBuffer algorithm converts file bytes into a unique SHA-256 hex string, solving critical data compliance risks.
2. Dynamic Match Evaluator: An array intersection algorithm normalizes candidate skills vs job requirements (O(N x M) time complexity), yielding a deterministic probability score that drives the platform's core gating mechanism.
3. O(1) On-chain Duplicate Detection: The smart contract utilizes packed keccak256 fingerprints to instantly reject duplicate credential mints without iteration.

Quality

The repository maintains a clean structure: distinct directories for contracts, frontend UI, JS services, serverless netlify/functions, and automated tests. Code avoids monolithic scripts by importing logic from discrete modules, ensuring high reusability and explicit naming conventions.